Endpoint Investigator

1. Problema e estratégia de investigação

Tecnologias Hackers - Avaliação Intermediária | 29/09/2026

O problema é relacionar informações de um endpoint Linux para identificar configurações que mereçam investigação. Processos, permissões e serviços constituem as três dimensões obrigatórias. Logs acrescentam contexto. O objetivo não é classificar automaticamente um host como comprometido: uma configuração de risco não demonstra que alguém a explorou.

A solução implementa uma ferramenta sob demanda em Python 3.10+, sem dependências externas em execução. Ela importa os artefatos do gerador fornecido ou coleta um snapshot Linux. O processamento é local e determinístico; não há modelo de linguagem na execução. A demonstração distribuída utiliza exclusivamente datasets sintéticos.

Fluxo e componentes

Etapa	Implementação	Saída
Coleta/importação	collect.py e input.py	Registros originais e origem
Normalização	input.py	Evidências tipadas com IDs
Correlação	engine.py	Relações com método e limite
Análise	Regras R01-R07	Observação, interpretação e hipótese
Apresentação	report.py	JSON, HTML e Markdown

No modo dataset, são lidos processes.csv, permissions.csv e services.txt; journal.log é opcional. README.txt, metadata.json e o nome do cenário não participam da análise. Isso impede que os rótulos didáticos determinem o resultado.

Na coleta local, são usados os comandos vistos em aula: ps -eo pid,ppid,user,stat,cmd fornece PID, PPID, usuário, estado e comando; systemctl list-units e systemctl status listam os serviços running e seu Main PID; journalctl -n fornece eventos. As permissões (dono, grupo e rwx, como no ls -l) são lidas para os caminhos de código identificados e seus diretórios. A coleta registra falhas de acesso, sem tratá-las como ausência de atividade.

Evidência estruturada antes da interpretação

Cada registro contém id, tipo, origem, bruto e dados. IDs P, A, S e L identificam processos, permissões, serviços e logs. A proveniência do dataset inclui arquivo e linha; na coleta local, inclui a fonte e o instante observado. Relações C e achados F referenciam esses registros por ID.

O JSON preserva dados brutos, normalização, relações, hipóteses e informações faltantes. SHA-256 identifica os bytes dos arquivos de entrada; não autentica a origem da coleta. Um registro pode ser conferido sem aceitar a interpretação associada. Os IDs são locais à análise, não identidades globais entre execuções.

2. Correlações e regras

São implementadas correlações entre serviço, processo e código; entre processo, PPID e identidade; e entre serviços/processos e logs. Na coleta local, o Main PID informado pelo systemctl status liga o serviço ao processo, e desse processo vêm o usuário e o comando do serviço. No dataset, argv completo e usuário iguais estabelecem apenas uma associação candidata. Correspondência por substring do nome do processo não é utilizada.

Regra central: relação de privilégio

R01 combina um serviço ativo configurado como root, ou um processo root associado, com um caminho de código referenciado e metadados que indicam escrita para outros ou propriedade de usuário não root. Quando não há processo correspondente, o achado explicita que o privilégio foi observado na configuração, sem confirmar execução root.

O teste de escrita usa a máscara octal 0002. A regra identifica uma relação potencialmente insegura: se um usuário não privilegiado conseguir alterar esse código e houver uso posterior sob privilégio, poderá influenciar a execução privilegiada. Os dados não estabelecem essas condições históricas nem demonstram exploração.

Critérios de investigação

Regra	Relação/condição	Resultado
R01	Serviço/processo root + código gravável por outros ou de proprietário não root	Risco; prioridade alta
R02	Código privilegiado + diretório pai imediato com escrita para outros	Risco; alta
R03	Arquivo gravável por outros sem vínculo privilegiado estabelecido	Configuração; média
R04	Arquivo root com SUID ou SGID e bit de execução	Informação para revisão
R05	Código privilegiado gravável por grupo; integrantes desconhecidos	Inconclusivo; média
R06	Serviço + descendente root + URL em argumento	Contexto informativo
R07	PPID relaciona filho root a pai não root	Inconclusivo; média

R02 existe porque quem pode escrever em um diretório pode apagar e recriar os arquivos dentro dele, mesmo que o script em si esteja protegido. R04 não trata SUID/SGID como falha: programas como passwd e sudo precisam desse bit; o achado apenas indica uma revisão, como a feita com find / -perm -4000 em aula [1].

R06 não verifica sockets. A existência de URL no comando pode corresponder a monitoramento, atualização ou outra operação autorizada. R07 registra diferença de identidades, sem diagnosticar escalada não autorizada. Logs associados por PID têm confiança baixa, pois o identificador pode ter sido reutilizado. A confiança refere-se à associação, não à probabilidade de ataque.

3. Evidências da demonstração

O comando python3 demo.py gera seis cenários e executa o motor. No cenário correlation, o achado F0001/R01 utiliza os registros abaixo, preservados em evidencias/correlation/evidencias.json. Os números de linha incluem o cabeçalho dos arquivos tabulares.

ID e origem	Informação observada
S0003 - services.txt:4	backup-agent.service, running, root; /bin/bash /opt/backup/backup.sh
P0004 - processes.csv:5	PID 2417, PPID 1, usuário root; mesmo comando do serviço
A0002 - permissions.csv:3	/opt/backup/backup.sh, arquivo, proprietário root, grupo root, modo 0777

Evidência. O serviço declara root, o processo candidato aparece como root e o modo do script concede escrita a outros. A associação serviço/processo usa argv completo e identidade nominal; o caminho absoluto associa o código aos metadados.

Interpretação. Um recurso de código referenciado por execução privilegiada apresenta permissão ampla. Trata-se de uma relação de privilégio que requer verificação. Os dados não mostram quem de fato consegue alterar o arquivo.

Hipótese. Caso um usuário não privilegiado possa alterar o script e o serviço utilize a versão alterada, esse usuário poderá influenciar código executado como root. Nenhuma modificação ou execução de conteúdo adulterado foi demonstrada.

Evidências ausentes. Integrantes dos grupos e permissões de todos os diretórios do caminho; conteúdo do script e comparação do hash SHA-256 com uma versão aprovada; logs que relacionem alteração não autorizada e execução. A prioridade alta ordena a investigação, sem declarar comprometimento.

Contexto adicional e controles

O processo P0005, PID 2421 e PPID 2417, contém uma URL no argv. A cadeia PPID relaciona esse processo ao contexto do serviço, gerando R06 informativo. A URL não prova uma conexão estabelecida. A0004 contém um arquivo root com modo 4755 e gera R04 informativo. Esses dois registros não são adicionados como provas de exploração ao achado central.

Os logs L0002-L0005 estão associados ao serviço por unidade/tag. São apresentados na ordem da fonte. O texto não informa ano nem fuso; a ferramenta não o transforma em cronologia absoluta nem interpreta proximidade temporal como causalidade. Um timestamp de observação de processo também não equivale ao seu instante de criação.

Como controle, trocar apenas o modo do script de 0777 para 0700 remove R01. O caso permission mantém somente R03, pois não estabelece consumidor privilegiado. O caso privileged_service executa como root com script protegido e não gera prioridade alta. O caso ambiguous permanece informativo.

4. Validação, decisões e limitações

Os resultados abaixo foram produzidos por execução, em Python 3.13 na VM Kali. O registro completo está em evidencias/demonstracao.json. A seed usada na demonstração é 42.

Cenário	Evidências	Relações	Alta / média / informativa
normal	21	16	0 / 0 / 0
permission	23	16	0 / 1 / 0
privileged_service	17	21	0 / 0 / 1
correlation	21	22	1 / 0 / 2
ambiguous	20	23	0 / 0 / 1
random	26	22	0 / 0 / 0

Testes. Foram executados 32 testes, sem falhas ou erros. Eles verificam controles negativos, alterações de permissões, diretório gravável, SUID/SGID, grupos, identidade, referências, fontes ausentes e HTML escapado. Um teste percorre 90 combinações de nível/seed. Os cinco cenários fixos do gerador original também são aceitos. Isso verifica comportamentos definidos; não estima taxas de detecção em hosts reais.

Coleta real. Na VM Kali, a coleta obteve 139 processos, 17 serviços ligados aos processos pelo Main PID, 81 registros de permissão e 50 linhas de log. Um achado R07 (fusermount3 root com pai não root) mostra o caso inconclusivo: é um utilitário SUID legítimo, e a ferramenta não o classifica como ataque.

Decisões. Biblioteca padrão, regras explícitas e relatório offline reduzem dependências. O gerador original foi preservado. Na cópia do projeto foram corrigidos formatos misturados no cenário aleatório, o rótulo permission e o ISO que indicava -03:00 sem converter a hora UTC. A opção --scenario permite repetir casos específicos.

Limitações. A coleta é uma foto do momento; processos podem terminar ou ter o PID reaproveitado. Só as permissões rwx do arquivo e do diretório pai são analisadas, sem verificar os integrantes dos grupos. A extração cobre executáveis absolutos e scripts diretos; não resolve bash -c nem caminhos relativos. Não há coleta de conexões de rede, análise de conteúdo nem prova de exploração. Ausência de achados não demonstra segurança.

Uso de IA. Houve apoio de ferramentas de IA (ChatGPT/Codex e Claude) na implementação, documentação e testes; os resultados foram revisados pelo grupo. Não há LLM na execução: a evidência estruturada precede qualquer interpretação.

Referências

[1] Handout da Aula 2, Investigando o sistema operacional (Prof. Rodolfo Avelino): ps, PID/PPID, permissões, SUID/SGID, systemctl e journalctl. [2] OccupyTheWeb, Linux Basics for Hackers, cap. 5 e 6. Enunciado e gerador: arquivos fornecidos pelo docente.